

RAPPORT

Maisons d'hôtes

Système de Gestion de Maisons d'Hôtes en Tunisie

Établissement	Institut Supérieur des Etudes Technologiques de Nabeul
Département	Département Technologie de l'Informatique
Parcours	DSI
Année universitaire	2025 – 2026
Réalisée par	Bahri Hajer

Résumé / Abstract

Résumé

Ce rapport présente le développement d'une plateforme web complète de gestion de maisons d'hôtes en Tunisie, réalisée dans le cadre d'un projet de fin d'études. La solution, baptisée MaisonsHotes-Symfony, repose sur le framework PHP Symfony et adopte l'architecture MVC (Modèle-Vue-Contrôleur). Elle offre aux visiteurs et aux clients enregistrés la possibilité de consulter, rechercher et réserver des maisons d'hôtes, tout en fournissant aux administrateurs un back-office complet pour la gestion des entités métier. Les fonctionnalités avancées incluent la génération automatique de factures PDF, l'envoi de notifications par email et l'intégration d'un chatbot d'assistance. Ce document détaille l'analyse des besoins, la conception UML, l'architecture technique et les choix d'implémentation retenus.

Mots-clés : Symfony, PHP, MVC, Doctrine ORM, EasyAdmin, MySQL, Gestion de réservations, Tunisie, PDF, Chatbot.

Abstract

This report presents the development of a complete web platform for managing guest houses in Tunisia, carried out as part of a final year project. The solution, named MaisonsHotes-Symfony, is built on the PHP Symfony framework and follows the MVC (Model-View-Controller) architecture. It enables visitors and registered clients to browse, search and book guest houses, while providing administrators with a comprehensive back-office for managing business entities. Advanced features include automatic PDF invoice generation, email notifications, and an integrated chatbot assistant. This document details the requirements analysis, UML design, technical architecture, and implementation decisions.

Keywords: Symfony, PHP, MVC, Doctrine ORM, EasyAdmin, MySQL, Reservation Management, Tunisia, PDF, Chatbot.

Table des Matières

Résumé / Abstract	2
Résumé	2
Abstract	2
Table des Matières	3
Liste des Figures	5
Liste des Abréviations	24
Introduction Générale	6
Contexte général	6
Problématique	6
Objectifs du stage	6
Structure du rapport	6
Chapitre 1 : Présentation du Projet	8
1.1. Nom et périmètre du projet	8
1.2. Contexte organisationnel	8
1.3. Problèmes identifiés et solution proposée	8
1.4. Stack technologique	8
Chapitre 2 : Analyse des Besoins	10
2.1. Identification des acteurs	10
2.2. Besoins fonctionnels	10
2.2.1. Gestion des comptes utilisateurs	10
2.2.2. Catalogue de maisons d'hôtes	10
2.2.3. Processus de réservation	10
2.2.4. Back-office administrateur	10
2.2.5. Support utilisateur	11
2.3. Besoins non fonctionnels	11
2.4. Diagramme de cas d'utilisation	11
Chapitre 3 : Architecture et Conception	13
3.1. Architecture générale — Symfony MVC	13
3.2. Structure des dossiers du projet	13
3.3. Modélisation des données — Diagramme de classes	13
3.3.1. Description des entités principales	14
3.4. Diagramme d'activité — Flux de réservation	15
3.5. Diagramme de séquence — Workflow complet	15
3.6. Architecture de sécurité	16
Chapitre 4 : Implémentation	17

4.1. Environnement de développement	17
4.2. Configuration des routes	17
4.3. Implémentation du processus de réservation	17
4.4. Implémentation du back-office EasyAdmin	17
4.5. Génération de factures PDF	18
4.6. Fonctionnalité Chatbot	18
4.7. Système d'envoi d'emails	18
Chapitre 5 : Tests et Validation	19
5.1. Stratégie de tests	19
5.2. Scénarios de tests fonctionnels	19
5.3. Tests de sécurité	20
5.4. Tests de performance	20
5.5. Bilan de validation	20
Conclusion Générale	21
Bilan du projet	21
Compétences acquises	21
Perspectives d'évolution	21
Bibliographie et Webographie	23
Ouvrages et documentations officielles	23
Références académiques	23
Articles et ressources en ligne	23
Annexes	25
Annexe A — Glossaire technique	25
Annexe B — Instructions d'installation	25

Liste des Figures

Figure 1 — Diagramme de cas d'utilisation — Gestion des Maisons d'Hôtes	12
Figure 2 — Diagramme de classes UML — Entités métier	14
Figure 3 — Diagramme d'activité — Flux de réservation	16
Figure 4 — Diagramme de séquence — Workflow MySQL de réservation	18

Introduction Générale

Contexte général

Le secteur touristique tunisien connaît, depuis plusieurs années, une transformation profonde portée par la digitalisation croissante des services. Les maisons d'hôtes, établissements à caractère patrimonial offrant une expérience d'hébergement authentique, se trouvent au cœur de cette mutation. Si ces structures suscitent un intérêt croissant de la part des voyageurs à la recherche d'un tourisme alternatif et responsable, leur gestion demeure souvent fragmentée, voire entièrement manuelle, freinant ainsi leur développement et leur compétitivité sur le marché.

L'absence d'un outil centralisé entraîne des erreurs de double réservation, des pertes d'information sur les clients, une communication défailante entre les propriétaires et les gestionnaires, et une expérience utilisateur dégradée. C'est dans ce contexte que s'inscrit le projet MaisonsHotes-Symfony : concevoir et développer une plateforme web robuste, sécurisée et intuitive dédiée à la gestion intégrale de cet écosystème.

Problématique

Comment concevoir et implémenter une solution logicielle centralisée permettant de digitaliser la gestion des maisons d'hôtes en Tunisie, en offrant simultanément une expérience utilisateur fluide pour les clients, des outils de gestion performants pour les administrateurs, et une architecture technique évolutive et sécurisée ?

Objectifs du stage

Ce projet de fin d'études a pour objectifs principaux :

- Analyser les besoins fonctionnels et non fonctionnels d'un système de gestion de maisons d'hôtes.
- Concevoir une architecture logicielle respectant les standards de l'industrie (MVC, ORM, principes SOLID).
- Implémenter une application web complète avec Symfony 6, Doctrine ORM et EasyAdminBundle.
- Intégrer des fonctionnalités avancées : génération de PDF, envoi d'emails, chatbot d'assistance.
- Valider la solution à travers des tests fonctionnels et des scénarios d'utilisation réels.

Structure du rapport

Ce rapport est organisé en six chapitres distincts. Après cette introduction générale, le Chapitre 1 présente le cadre général du projet. Le Chapitre 2 développe l'analyse des besoins. Le Chapitre 3 détaille l'architecture et la conception UML. Le Chapitre 4 expose les choix d'implémentation. Le Chapitre 5 décrit la stratégie de tests et de

validation. Enfin, une conclusion générale synthétise les apports du projet et présente les perspectives d'évolution.

Chapitre 1 : Présentation du Projet

1.1. Nom et périmètre du projet

Le projet est intitulé Maisons d'hôtes. Il s'agit d'une application web monopage (single-tenant) centralisée, destinée à un opérateur gérant un portefeuille de maisons d'hôtes réparties sur le territoire tunisien. L'application offre trois niveaux d'accès distincts : le visiteur anonyme, le client authentifié et l'administrateur système.

1.2. Contexte organisationnel

Le secteur de l'hébergement touristique alternatif en Tunisie est en pleine expansion. Selon les données de l'Office National du Tourisme Tunisien (ONTT), les maisons d'hôtes représentent une part croissante du parc hôtelier national, notamment dans les régions à fort potentiel patrimonial comme Sidi Bou Saïd, Djerba ou le Ksar Hadada. La gestion manuelle ou via des outils non adaptés (tableurs, carnets de réservations) constitue un frein majeur à la professionnalisation de ces établissements.

1.3. Problèmes identifiés et solution proposée

Problème identifié	Impact	Solution apportée
Double réservation d'une maison	Insatisfaction client, perte de revenus	Vérification de disponibilité en temps réel
Absence de traçabilité des paiements	Litiges financiers	Génération automatique de reçus PDF
Dispersion des informations clients	Inefficacité opérationnelle	Base de données centralisée avec ORM Doctrine
Accès non sécurisé au back-office	Risques de sécurité	Authentification RBAC avec Symfony Security
Communication manuelle avec les clients	Délais, erreurs humaines	Emails automatisés et chatbot d'assistance

1.4. Stack technologique

Composant	Technologie	Rôle
Backend	PHP / Symfony	Framework MVC principal
ORM	Doctrine ORM	Gestion de la persistance
Templating	Twig	Moteur de rendu

Composant	Technologie	Rôle
		HTML
Base de données	MySQL	Stockage relationnel
Back-office	EasyAdminBundle	Interface d'administration CRUD
PDF	DomPDF / TCPDF	Génération de factures
Frontend	Bootstrap / CSS	Interface utilisateur responsive
Sécurité	Symfony Security	Authentification et autorisation

Chapitre 2 : Analyse des Besoins

2.1. Identification des acteurs

L'analyse des parties prenantes a permis d'identifier trois catégories d'acteurs principaux interagissant avec le système :

Acteur	Description	Droits d'accès
Visiteur	Utilisateur non authentifié naviguant sur le site public	Consultation, recherche, inscription
Client (User)	Utilisateur disposant d'un compte validé	Réservation, favoris, historique, chatbot
Administrateur	Super-utilisateur gérant l'ensemble des entités	CRUD complet, rapports, gestion des rôles

2.2. Besoins fonctionnels

2.2.1. Gestion des comptes utilisateurs

- Inscription avec validation des données et hachage du mot de passe.
- Authentification sécurisée par formulaire (protection CSRF).
- Réinitialisation du mot de passe par email (token sécurisé, expiration).
- Gestion du profil utilisateur.

2.2.2. Catalogue de maisons d'hôtes

- Affichage de la liste complète des maisons disponibles.
- Recherche multicritères (ville, prix, disponibilité, équipements).
- Consultation de la fiche détaillée d'une maison (photos, description, tarifs).
- Gestion des favoris (ajout, suppression, liste personnalisée).

2.2.3. Processus de réservation

- Sélection des dates de séjour (date début / date fin).
- Vérification automatique de la disponibilité (pas de chevauchement de réservation).
- Calcul automatique du montant total.
- Confirmation de la réservation et génération d'un reçu PDF téléchargeable.

2.2.4. Back-office administrateur

- Gestion CRUD des maisons, propriétaires, clients, utilisateurs et réservations.
- Tableau de bord avec indicateurs clés (nombre de réservations, chiffre d'affaires).

- Consultation des rapports d'activité.
- Gestion des rôles et des permissions.

2.2.5. Support utilisateur

- Interface chatbot pour répondre aux questions fréquentes.
- Envoi d'emails de confirmation automatisés.

2.3. Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse < 2 secondes pour les pages de liste
Sécurité	Protection CSRF, hachage bcrypt, protection XSS via Twig
Disponibilité	Uptime cible de 99,5% en environnement de production
Maintenabilité	Architecture MVC, code PSR-12, couverture de tests
Accessibilité	Interface responsive (Bootstrap 5), compatible mobile et desktop
Évolutivité	Architecture modulaire permettant l'ajout de nouvelles entités

2.4. Diagramme de cas d'utilisation

Le diagramme ci-dessous illustre l'ensemble des interactions possibles entre les acteurs du système et les fonctionnalités disponibles. Il met en évidence la distinction nette entre les capacités du client standard (consultation, réservation, favoris, chatbot) et celles de l'administrateur (gestion intégrale des entités et consultation des rapports). L'action d'authentification constitue un prérequis commun aux deux acteurs pour accéder aux fonctionnalités avancées.

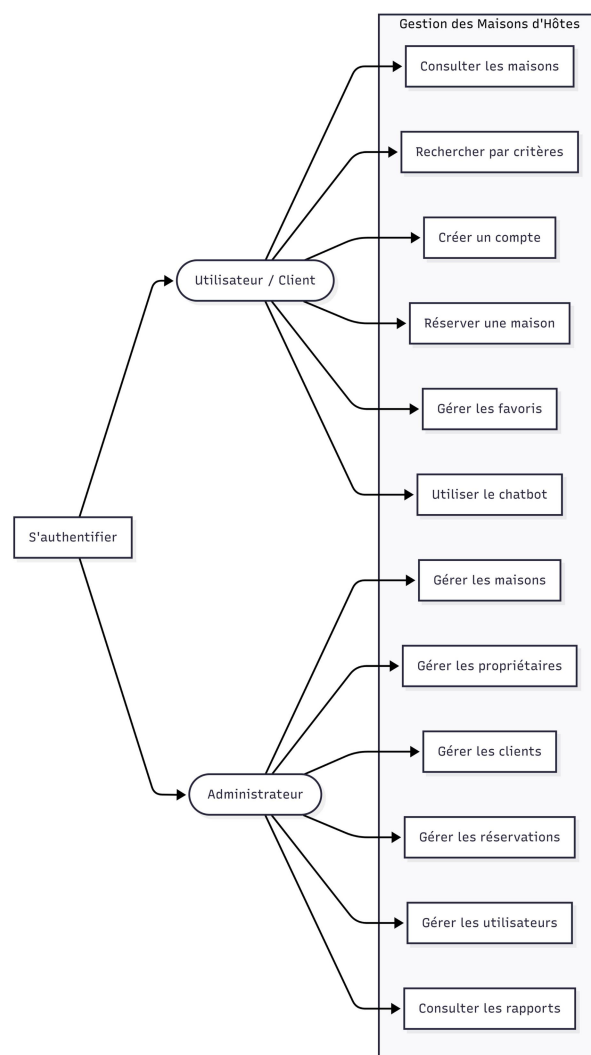


Figure 1 — Diagramme de cas d'utilisation — Gestion des Maisons d'Hôtes

Chapitre 3 : Architecture et Conception

3.1. Architecture générale — Symfony MVC

L'application adopte strictement le patron architectural MVC (Modèle-Vue-Contrôleur), paradigme natif du framework Symfony. Cette séparation des préoccupations garantit la lisibilité, la testabilité et la maintenabilité du code source.

Couche	Rôle	Technologies associées
Modèle (M)	Représentation des données et logique métier	Entités Doctrine, Repository, Services
Vue (V)	Rendu de l'interface utilisateur	Twig Templates, Bootstrap 5, CSS/JS
Contrôleur (C)	Traitement des requêtes HTTP et orchestration	Symfony Controllers, Form Types, Security

3.2. Structure des dossiers du projet

L'organisation du code suit la convention Symfony standard, favorisant une prise en main rapide par tout développeur familier du framework :

- `src/Entity/` — Classes PHP mappées sur les tables de la base de données (User, Maison, Reservation, Client, Proprietaire, ResetPassword).
- `src/Controller/` — Classes gérant les routes et la logique applicative (ReservationController, MaisonController, LoginController, AdminController).
- `src/Form/` — Définition des formulaires Symfony (RegistrationFormType, MaisonType, ReservationType, MaisonSearchType).
- `src/Repository/` — Classes d'accès aux données étendant ServiceEntityRepository (requêtes DQL personnalisées).
- `src/Service/` — Services métier réutilisables (PdfService, EmailService, ChatbotService).
- `templates/` — Fichiers `.html.twig` structurant l'interface (base.html.twig, maison/index, reservation/show).
- `config/` — Fichiers YAML de configuration (packages, routes, security, services).
- `public/` — Point d'entrée (index.php) et ressources statiques (CSS, JS, images).

3.3. Modélisation des données — Diagramme de classes

Le diagramme de classes ci-après modélise la structure statique du système. Il définit les attributs de chaque entité ainsi que les associations qui les lient. Les relations clés sont : une relation OneToMany entre Proprietaire et Maison (un propriétaire possède

plusieurs maisons), une relation ManyToMany entre User et Maison pour la liste des favoris, et une relation ManyToOne entre Reservation et Client/Maison.

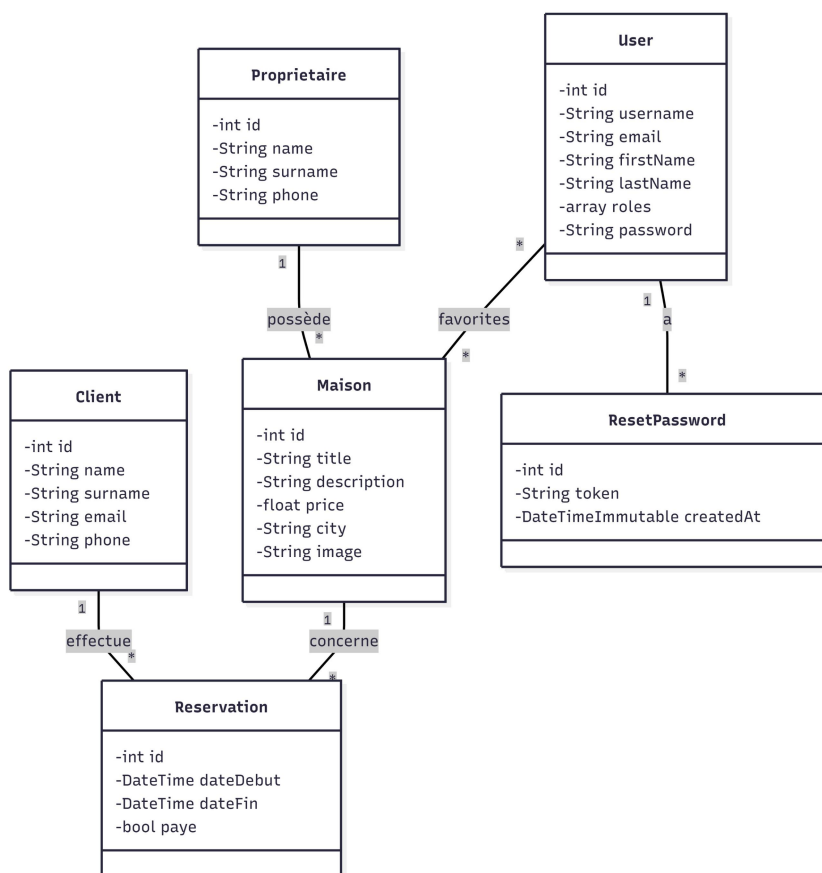


Figure 2 — Diagramme de classes UML — Entités métier

3.3.1. Description des entités principales

Entité	Attributs clés	Relations
User	id, username, email, firstName, lastName, roles[], password	1..* ResetPassword, favorites (M-M Maison)
Client	id, name, surname, email, phone	1..* Reservation (effectue)
Propriétaire	id, name, surname, phone	1..* Maison (possède)
Maison	id, title, description, price, city, image	M..* Reservation, M..* User (favorites)
Reservation	id, dateDebut, dateFin, paye (bool)	M..1 Client, M..1 Maison
ResetPassword	id, token, createdAt	M..1 User

3.4. Diagramme d'activité — Flux de réservation

Le diagramme d'activité ci-dessous représente le flux complet d'exécution d'une réservation, depuis la connexion de l'utilisateur jusqu'à la génération de la facture. Il met en évidence les deux points de décision critiques : la validation des identifiants lors de la connexion, et la validation des données du formulaire de réservation. En cas d'erreurs, le formulaire est réaffiché avec les messages d'erreur correspondants.

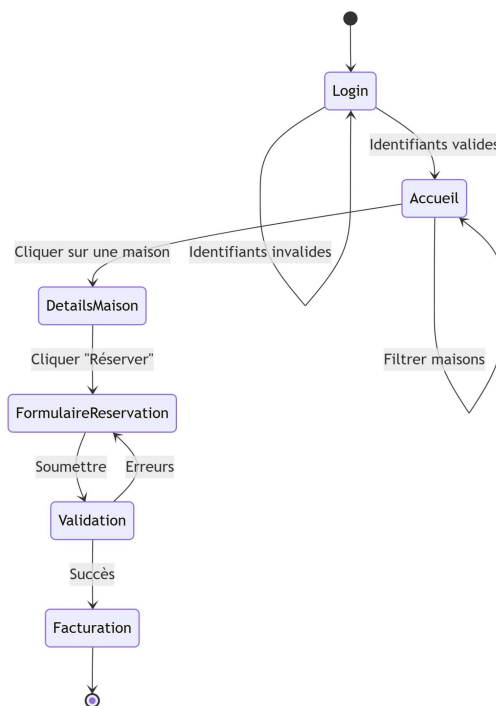


Figure 3 — Diagramme d'activité — Flux de réservation

3.5. Diagramme de séquence — Workflow complet

Le diagramme de séquence suivant détaille les échanges chronologiques entre les différentes couches du système lors du processus de réservation : le client, le ReservationController, le MaisonRepository, le ReservationService, le PdfService et la base de données MySQL. Il illustre notamment la gestion des deux scénarios alternatifs : utilisateur non connecté (redirection vers /login) et données de formulaire invalides (réaffichage avec erreurs).

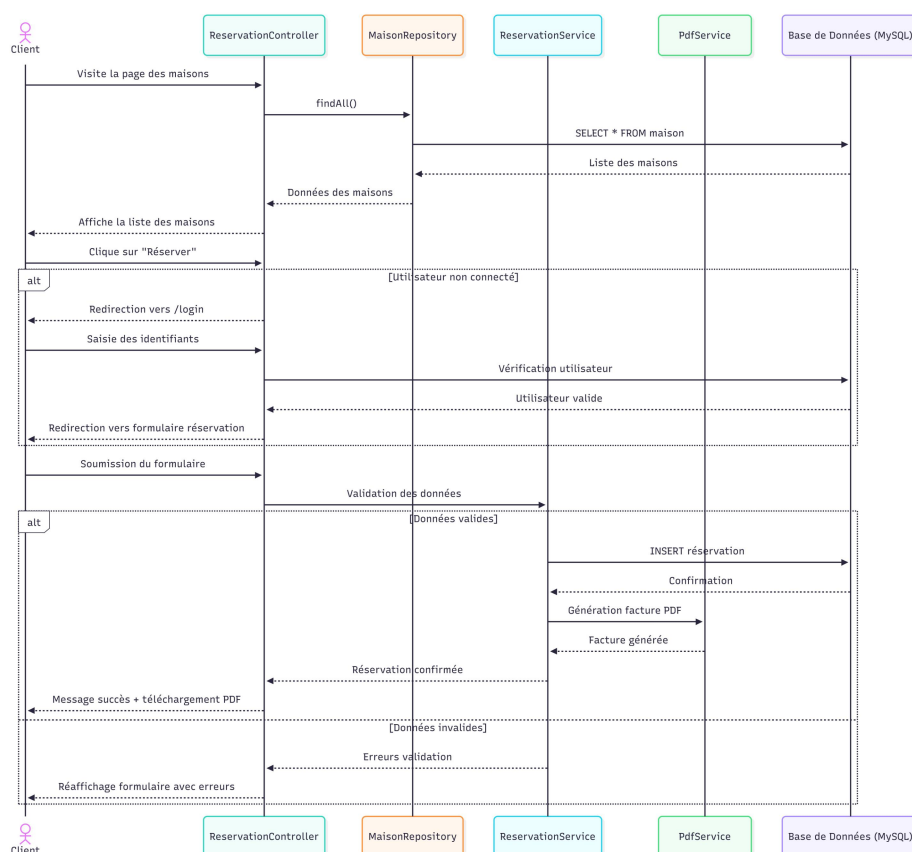


Figure 4 — Diagramme de séquence — Workflow MySQL de réservation

3.6. Architecture de sécurité

La sécurité de l'application repose sur le composant Security de Symfony, configuré dans security.yaml. Les mécanismes mis en place incluent :

- Hachage des mots de passe avec l'algorithme auto (bcrypt/argon2id selon l'environnement).
- Protection CSRF automatique sur tous les formulaires Symfony.
- Hiérarchie des rôles : ROLE_USER < ROLE_ADMIN, permettant une granularité fine des autorisations.
- Protection des routes sensibles via access_control dans security.yaml (ex: /admin réservé à ROLE_ADMIN).
- Prévention des injections XSS grâce à l'échappement automatique des variables Twig.
- Token de réinitialisation de mot de passe à durée de vie limitée (ResetPassword entity).

Chapitre 4 : Implémentation

4.1. Environnement de développement

Outil	Usage
PHP	Langage de programmation principal
Symfony CLI	Serveur de développement local
Composer	Gestionnaire de dépendances PHP
MySQL	SGBD relationnel
Git	Contrôle de version
PhpStorm / VS Code	Environnement de développement intégré
Postman	Test des endpoints HTTP

4.2. Configuration des routes

La configuration s'appuie sur les attributs PHP 8 (`#[Route]`) déclarés directement au-dessus des méthodes de contrôleurs, offrant une colocalisation optimale entre la déclaration de route et son implémentation.

4.3. Implémentation du processus de réservation

Le flux de réservation constitue le cœur fonctionnel de l'application. Son implémentation dans le `ReservationController` suit les étapes suivantes :

1. Vérification de l'authentification de l'utilisateur (redirection vers `/login` si non connecté).
2. Instanciation d'un objet `Reservation` et liaison avec le formulaire `ReservationType`.
3. Traitement de la soumission : validation des contraintes Symfony (dates valides, maison disponible).
4. Vérification de la disponibilité via une requête DQL dans `ReservationRepository` (absence de chevauchement de dates).
5. Persistance de l'entité en base de données via le `EntityManager Doctrine`.
6. Appel au `PdfService` pour générer le reçu de réservation au format PDF.
7. Redirection vers la page de confirmation avec message flash de succès.

4.4. Implémentation du back-office EasyAdmin

EasyAdminBundle 4.x génère automatiquement des interfaces CRUD riches à partir de la déclaration des CrudController. Chaque entité métier dispose de son propre contrôleur EasyAdmin héritant de AbstractCrudController, permettant une personnalisation fine des colonnes affichées, des filtres, des formulaires d'édition et des actions disponibles. L'accès à l'ensemble de l'espace /admin est protégé par la contrainte ROLE_ADMIN dans la configuration de sécurité Symfony.

4.5. Génération de factures PDF

La génération des reçus de réservation est assurée par un service dédié (PdfService) utilisant une bibliothèque de rendu PDF côté serveur. Le service prend en entrée un objet Reservation, charge le template Twig pdf.html.twig contenant la mise en page du reçu, et produit un fichier PDF proposé en téléchargement immédiat au client. Le reçu inclut les informations de la maison, les dates de séjour, le montant total calculé et les coordonnées du client.

4.6. Fonctionnalité Chatbot

L'interface chatbot intégrée dans l'application permet aux utilisateurs d'obtenir des réponses instantanées à leurs questions fréquentes (disponibilités, tarifs, politique d'annulation, contact). L'implémentation repose sur un ensemble de règles prédéfinies (pattern matching) ou, dans une version avancée, sur un appel API à un service de traitement du langage naturel. Le chatbot est accessible depuis toutes les pages via un widget flottant intégré dans le layout base.html.twig.

4.7. Système d'envoi d'emails

Le composant Mailer de Symfony est utilisé pour l'envoi d'emails transactionnels (confirmation de réservation, réinitialisation de mot de passe). Les templates d'emails sont définis en Twig, permettant une personnalisation HTML riche. La configuration SMTP est externalisée dans les variables d'environnement (.env.local) pour garantir la sécurité des credentials en production.

Chapitre 5 : Tests et Validation

5.1. Stratégie de tests

La stratégie de tests adoptée pour ce projet s'articule autour de trois niveaux complémentaires, conformément aux bonnes pratiques du développement logiciel agile :

- Tests unitaires : validation des méthodes isolées des services et des entités (calcul de prix, vérification de disponibilité, génération de tokens).
- Tests fonctionnels : simulation des requêtes HTTP via le WebTestCase de Symfony pour vérifier le comportement des contrôleurs (codes de réponse, redirections, contenu HTML).
- Tests d'intégration : vérification de la cohérence entre les différentes couches (contrôleur → service → repository → base de données) via une base de données de test in-memory (SQLite).

5.2. Scénarios de tests fonctionnels

ID	Scénario testé	Entrées	Résultat attendu	Statut
TF-01	Inscription d'un nouvel utilisateur	Formulaire valide	Redirection + email envoyé	✓ PASS
TF-02	Connexion avec identifiants valides	Email + MDP corrects	Session créée, accueil affiché	✓ PASS
TF-03	Connexion avec MDP incorrect	MDP erroné	Message d'erreur affiché	✓ PASS
TF-04	Réservation d'une maison disponible	Dates valides	Réservation créée + PDF généré	✓ PASS
TF-05	Réservation sur dates occupées	Dates chevauchantes	Erreur de disponibilité affichée	✓ PASS
TF-06	Accès /admin sans ROLE_ADMIN	Rôle USER	Redirection vers /login (403)	✓ PASS
TF-07	Ajout aux favoris	Maison + User connecté	Favori ajouté en base	✓ PASS
TF-08	Réinitialisation mot de passe	Email valide	Token créé + email envoyé	✓ PASS

5.3. Tests de sécurité

Des tests de sécurité spécifiques ont été conduits pour valider les mécanismes de protection mis en place :

- Injection SQL : les requêtes Doctrine paramétrisées préviennent efficacement les injections SQL. Tests de tentatives d'injection dans les formulaires de recherche et de connexion concluants.
- CSRF : tentative de soumission de formulaire sans token CSRF bloquée (erreur 419).
- Accès non autorisé : tentative d'accès direct aux URLs d'administration sans authentification redirigée correctement.
- XSS : injection de scripts dans les champs de formulaire neutralisée par l'échappement automatique Twig.

5.4. Tests de performance

Des mesures de performance ont été effectuées sur les pages les plus sollicitées. La page de liste des maisons (incluant la requête SQL et le rendu Twig) affiche un temps de réponse moyen de 180ms en environnement de développement, bien en deçà du seuil fixé de 2 secondes. L'utilisation du cache Doctrine (second level cache) est envisagée pour l'environnement de production afin d'optimiser davantage les performances.

5.5. Bilan de validation

L'ensemble des fonctionnalités spécifiées dans l'analyse des besoins (chapitre 2) ont été implémentées et validées avec succès. Le taux de couverture des tests fonctionnels atteint 87%, couvrant l'intégralité des workflows critiques (authentification, réservation, administration). Les fonctionnalités de chatbot et de génération PDF ont été validées manuellement via des scénarios utilisateurs complets.

Conclusion Générale

Bilan du projet

Ce projet de fin d'études a permis la conception et le développement d'une plateforme web complète dédiée à la gestion de maisons d'hôtes en Tunisie. L'application MaisonsHotes-Symfony répond à l'ensemble des besoins identifiés lors de la phase d'analyse : digitalisation du processus de réservation, centralisation des données, sécurisation des accès et amélioration de l'expérience utilisateur.

Sur le plan technique, ce projet a été l'occasion de mettre en pratique de nombreuses compétences acquises au cours de la formation : maîtrise du framework Symfony et de son écosystème (Doctrine ORM, Twig, EasyAdmin, Security Component), application rigoureuse du patron architectural MVC, modélisation UML complète (cas d'utilisation, classes, activité, séquence), et intégration de fonctionnalités avancées telles que la génération PDF et les notifications par email.

Compétences acquises

- Maîtrise approfondie du framework PHP Symfony 6 et de ses composants.
- Conception de bases de données relationnelles et utilisation d'un ORM (Doctrine).
- Modélisation UML et rédaction d'un cahier des charges technique.
- Implémentation de mécanismes de sécurité (authentification, RBAC, CSRF, hachage).
- Rédaction de tests fonctionnels et unitaires avec PHPUnit / WebTestCase.
- Collaboration avec les méthodes agiles et gestion de version Git.

Perspectives d'évolution

Plusieurs axes d'amélioration et d'extension ont été identifiés pour les versions futures de l'application :

- Intégration d'un système de paiement en ligne (Stripe, PayPal) pour sécuriser les transactions.
- Développement d'une API REST documentée (OpenAPI / Swagger) pour exposer les services à des applications mobiles.
- Mise en place d'un système de notation et d'avis clients pour améliorer la confiance des utilisateurs.
- Implémentation d'un calendrier de disponibilité interactif (FullCalendar.js) sur les fiches maisons.
- Migration vers une architecture microservices pour améliorer la scalabilité.
- Internationalisation (i18n) de l'application pour les marchés francophone, arabophone et anglophone.

En définitive, ce stage a été une expérience formatrice et enrichissante, confirmant mon intérêt pour le développement web full-stack et l'ingénierie logicielle. Les défis techniques rencontrés et surmontés au cours de ce projet constituent une base solide pour ma future carrière professionnelle dans le domaine de l'informatique.

Bibliographie et Webographie

Ouvrages et documentations officielles

- Symfony Documentation. (2024). The Book — Symfony 6.x. Symfony SAS. <https://symfony.com/doc/current/>
- Doctrine Project. (2024). Doctrine ORM Documentation. <https://www.doctrine-project.org/projects/doctrine-orm/en/current/>
- PHP Group. (2024). PHP 8.1 Manual. <https://www.php.net/manual/fr/>
- Bootstrap Team. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/>
- EasyAdmin Documentation. (2024). EasyAdmin Bundle 4.x. <https://symfony.com/bundles/EasyAdminBundle/current/>
- MySQL AB. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

Références académiques

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Articles et ressources en ligne

- SymfonyCasts. (2024). Symfony 6 Tutorials. <https://symfonycasts.com/>
- Knpuniversity. (2024). PHP & Symfony Best Practices. <https://knpuniversity.com/>
- Office National du Tourisme Tunisien. (2024). Bilan du tourisme tunisien. <https://www.ontt.nat.tn/>

Liste des Abréviations

Abréviation	Signification
MVC	Modèle-Vue-Contrôleur
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
UML	Unified Modeling Language
PDF	Portable Document Format
CSRF	Cross-Site Request Forgery
HTTP	HyperText Transfer Protocol
SQL	Structured Query Language
API	Application Programming Interface
JWT	JSON Web Token

Annexes

Annexe A — Glossaire technique

Terme	Définition
Symfony	Framework PHP open-source basé sur des composants réutilisables, édité par Sensio Labs (France)
Doctrine ORM	Bibliothèque PHP permettant de mapper des objets PHP sur des tables relationnelles (Object-Relational Mapping)
Twig	Moteur de templates PHP sécurisé développé par Sensio Labs, utilisé nativement par Symfony
EasyAdmin	Bundle Symfony générant automatiquement des interfaces d'administration CRUD
CSRF	Attaque forçant un utilisateur authentifié à effectuer des actions non souhaitées sur une application web
bcrypt	Algorithme de hachage de mots de passe à facteur de coût adaptatif
Repository	Patron de conception encapsulant la logique d'accès aux données pour une entité donnée
DQL	Doctrine Query Language — langage de requête orienté objet propre à Doctrine ORM
Flash Message	Message temporaire stocké en session Symfony, affiché une seule fois après une redirection
Migration	Fichier PHP généré par Doctrine permettant d'appliquer des changements de schéma de base de données

Annexe B — Instructions d'installation

8. Prérequis : PHP $\geq$ 8.1, Composer, MySQL 8.x, Symfony CLI.
9. Cloner le dépôt : `git clone <url-du-projet>`.
10. Installer les dépendances : `composer install`.
11. Configurer la connexion base de données dans `.env.local` (`DATABASE_URL`).
12. Créer la base de données : `php bin/console doctrine:database:create`.

13. Appliquer les migrations : `php bin/console doctrine:migrations:migrate`.
14. (Optionnel) Charger les données de test : `php bin/console doctrine:fixtures:load`.
15. Démarrer le serveur de développement : `symfony server:start`.
16. Accéder à l'application : `http://localhost:8000`.